

SmartLocker

System

.NET on Linux Raspberry Pi
On-Premise Kiosk Application

RESEARCH PROJECT

Joint Venture Between

ISense ASV Sdn Bhd

and

UKM FTSM

July 2026

v1.0.0 - Final Release

PROJECT OVERVIEW

Objectives, Scope & Technology

OBJECTIVE

- ▶ Build a comprehensive locker management system for on-premise deployment on Raspberry Pi
- ▶ Support item borrowing, locker access control, and QR unlock functionality
- ▶ Provide role-based web interfaces for Admin and Staff management

TECHNOLOGY STACK

- ▶ Backend: ASP.NET Core 8.0
- ▶ Frontend: Razor Pages with Bootstrap
- ▶ Database: SQLite with Entity Framework Core
- ▶ Deployment: Raspberry Pi OS 64-bit

SCOPE

- ▶ Item inventory management with categorization
- ▶ Request approval workflow with QR token generation
- ▶ Borrow/return lifecycle with overdue tracking
- ▶ Audit logging and system monitoring

DEPLOYMENT

- ▶ Local network touchscreen kiosk interface
- ▶ Admin web portal for system management
- ▶ Staff web portal for requests and borrows
- ▶ Systemd service for auto-start on boot

✓ Complete & Ready for Demo

By Zara
Researcher/
Requirement Analyst

SYSTEM ACTORS & ROLES

User Roles and Responsibilities

ADMIN



- ▶ Full system control
- ▶ User management
- ▶ Item management
- ▶ Request approval/rejection
- ▶ Overdue tracking
- ▶ Audit log viewing
- ▶ System configuration

STAFF



- ▶ Search items
- ▶ Submit requests
- ▶ View active borrows
- ▶ Return items
- ▶ Extend loans
- ▶ View own history
- ▶ Limited dashboard

LOCKER KIOSK



- ▶ Public interface
- ▶ QR code scanning
- ▶ Unlock lockers
- ▶ Item returns
- ▶ No authentication
- ▶ Touchscreen optimized
- ▶ Audit logging

15 USE CASES IMPLEMENTED

Complete Coverage of System Functionality

- ▶ **UC-01** Login
- ▶ **UC-02** Dashboard
- ▶ **UC-03** Manage User
- ▶ **UC-04** Manage Item
- ▶ **UC-05** View Logs
- ▶ **UC-06** Manage Borrow
- ▶ **UC-07** Return Item
- ▶ **UC-08** View Overdue
- ▶ **UC-09** Manage Request
- ▶ **UC-10** Approve Request
- ▶ **UC-11** Reject Request
- ▶ **UC-12** Search Item
- ▶ **UC-13** Request Item
- ▶ **UC-14** Borrow Item
- ▶ **UC-15** Extend Loan

✓ All Use Cases Complete

DATABASE ARCHITECTURE

Entity Models & Schema Design

USER MANAGEMENT

- ▶ User
- ▶ Role

INVENTORY

- ▶ Item
- ▶ Category
- ▶ ItemStatus

LOCKER MGMT

- ▶ Locker
- ▶ LockerStatus

WORKFLOW

- ▶ Request
- ▶ RequestStatus
- ▶ Borrow
- ▶ BorrowStatus

SECURITY

- ▶ LockerAccessToken

AUDIT

- ▶ SystemLog

TECHNOLOGY

- ▶ Database: SQLite
- ▶ ORM: Entity Framework Core
- ▶ Relationships: Fully normalized with foreign keys

By Abu
Backend
Developer

BACKEND ARCHITECTURE

Service Layer & Business Logic

AuthenticationService

User login, password hashing, verification

UserService

User CRUD, role management, enable/disable

ItemService

Item management, search, status updates

BorrowService

Borrow lifecycle, overdue tracking, extension

RequestService

Request creation, approval, rejection

LockerService

Locker management, QR token generation

LogService

System audit logging with filtering

QrCodeService

QR code generation and validation

LockerHardwareService

Hardware abstraction (Mock & GPIO)

9 Service Classes • Fully Decoupled • Testable

By Abu
Backend
Developer

AUTHENTICATION & AUTHORIZATION

Security Controls

Password Security

BCrypt hashing with salt, no plaintext storage

Authorization

Role-based access control verified

QR Token Security

Cryptographically secure, single-use, time-limited

Input Validation

All forms validated server-side

Anti-CSRF

Anti-forgery tokens on all POST operations

Audit Logging

All actions logged with user, action, resource, timestamp

Error Handling

No sensitive data exposed in error messages

✓ All Controls Implemented

ADMIN DASHBOARD

Admin Interface Overview

Dashboard

- ▶ System statistics, quick navigation

Manage Users

- ▶ Create, edit, disable users with role assignment

Manage Items

- ▶ Create items, assign to lockers, categorize

Manage Requests

- ▶ View pending requests, approve/reject with QR generation

Manage Borrows

- ▶ View active borrows, return items, extend loans

View Overdue

- ▶ Track overdue items, mark as lost

View Logs

- ▶ Audit trail with filtering by action, resource, user, date

7 Admin Pages • Full Control • Responsive

STAFF INTERFACE

Staff Portal Features

Dashboard

- ▶ Request and borrow statistics, quick links

Search Items

- ▶ Filter by category, status, availability

Request Item

- ▶ Submit requests with justification

My Borrows

- ▶ View active borrows with due dates

Return Item

- ▶ Return borrowed items with confirmation

Extend Loan

- ▶ Request loan extensions with new due dates

Responsive Design

- ▶ Works on desktop and tablet

4 Staff Pages • User-Friendly • Mobile-Ready

LOCKER KIOSK INTERFACE

Touchscreen Kiosk UI

Home Screen

- ▶ Welcome message, instructions, navigation

QR Unlock

- ▶ Scan or paste QR token for locker access

Success Screen

- ▶ Confirmation with item details and next steps

Error Screen

- ▶ Clear error messages for failed attempts

Touchscreen Optimization

- ▶ Large buttons (44x44px minimum), readable fonts

Responsive Layout

- ▶ Optimized for 1024x600 kiosk display

No Login Required

- ▶ Public access for item returns and unlocks

3 Kiosk Pages • Public Access • Touch-Optimized

REQUEST APPROVAL WORKFLOW

End-to-End Request Approval Flow

Step 1

Staff submits request for item

Step 2

Request stored with "Pending" status

Step 3

Admin reviews pending requests

Step 4

Admin approves request

Step 5

Borrow record automatically created

Step 6

Item status changed to "Borrowed"

Step 7

Locker status changed to "Occupied"

Step 8

QR access token generated (10-minute expiry)

Step 9

Token displayed to staff for kiosk unlock

9 Steps • Fully Automated • Secure

QR UNLOCK & HARDWARE INTEGRATION

QR Token Security & Hardware Control

Token Generation

Cryptographically secure RNG (256-bit)

Token Expiry

10-minute window

Single-Use

Token marked as used after first validation

Failed Attempt Lockout

5 failed attempts trigger lockout

Hardware Abstraction

ILockerHardwareService interface

Mock Mode

Simulates hardware for development

GPIO Mode

Real relay control on Raspberry Pi

Logging

All unlock attempts logged for audit trail

✓ Secure • ✓ Flexible • ✓ Auditable

By Abu
Backend
Developer

BORROW & RETURN WORKFLOW

Item Borrow & Return Process

Borrow Creation

Automatic on request approval

Status Updates

Item → Borrowed, Locker → Occupied

Return Process

Staff clicks "Return" on active borrow

Status Restoration

Item → Available, Locker → Available

Loan Extension

Staff can extend due date before expiry

Overdue Tracking

System calculates days overdue

Lost Item Handling

Admin can mark items as lost

Database Transactions

All operations wrapped in transactions for consistency

✓ Automated • ✓ Consistent • ✓ Reliable

DEPLOYMENT ARCHITECTURE

Raspberry Pi Deployment Strategy

Hardware

Raspberry Pi 4/5 with 2GB+ RAM

OS

Raspberry Pi OS 64-bit

Runtime

.NET 8.0 Runtime

Service

systemd service for auto-start and restart

Database

SQLite at /var/lib/smartlocker/smartlocker.db

User

Dedicated 'smartlocker' system user (no root)

Kiosk

Chromium in fullscreen kiosk mode

mDNS

Optional hostname setup (smartlocker.local)

✓ Production-Ready • ✓ Secure • ✓ Scalable

DEPLOYMENT SCRIPTS

Automated Deployment & Installation

publish-linux-arm64.sh

Publishes app for linux-arm64 architecture

install-raspberrypi.sh

Automated installation on Raspberry Pi — Updates system packages, Installs .NET 8.0 Runtime, Creates directories and permissions, Sets up systemd service, Configures application user

smartlocker.service

systemd unit file for auto-start and automatic restart on failure

chromium-kiosk-autostart.sh

Kiosk mode boot configuration for Chromium fullscreen display

4 Scripts • Fully Automated • Zero-Touch Deployment

SECURITY REVIEW & FINDINGS

Security Controls & Compliance

✓ Password Security

BCrypt hashing, no plaintext storage

✓ Authorization

Role-based access control verified

✓ QR Token Security

Cryptographically secure, single-use, time-limited

✓ Input Validation

All forms validated server-side

✓ Anti-CSRF

Anti-forgery tokens on all POST operations

✓ Audit Logging

All actions logged with user, action, resource, timestamp

✓ Error Handling

No sensitive data exposed in error messages

7/7 Controls Verified • Production-Ready

BUG INVESTIGATION & EDGE CASES

Edge Case Handling & Risk Mitigation

✓ Double Approval Prevention

Check if already approved

✓ Double Return Prevention

Check if already returned

✓ Duplicate Request Prevention

One active request per user/item

✓ Item Availability Check

Prevent borrowing unavailable items

✓ Locker Availability Check

Prevent assigning to unavailable lockers

✓ Expired QR Handling

Reject expired tokens with clear message

✓ GPIO Failure Handling

Mock mode allows testing without hardware

✓ Race Condition Mitigation

Database transactions ensure consistency

8/8 Edge Cases Handled • Robust System

UAT & USER ACCEPTANCE TESTING

Testing & Quality Assurance Results

✓ All 15 Use Cases

Tested and validated

✓ All 8 End-to-End Flows

Passed successfully

✓ Admin Screens

Dashboard, Users, Items, Requests, Borrows, Overdue, Logs

✓ Staff Screens

Dashboard, Search, Request, Borrows

✓ Kiosk Screens

Home, QR Unlock, Return

✓ Responsive Design

Desktop, tablet, mobile, kiosk

✓ Error Handling

Graceful error messages, no crashes

✓ Performance

All pages load < 3 seconds

✓ UAT PASSED • Ready for Production

PROJECT DELIVERABLES

Complete Project Deliverables

Source Code

156 files, fully functional ASP.NET Core application

Documentation

7 phase reports, deployment guide, security review, UAT checklist

Scripts

Publish, install, systemd service, kiosk setup

Database

15 tables, migrations, seed data

UI

16 pages (Admin, Staff, Locker, Auth)

Services

9 service classes with complete business logic

Models

13 entity models with relationships

GitHub Repository

Public repository with all code and documentation

✓ Complete • ✓ Documented • ✓ Production-Ready

By Ali
Fullstack
Integrator

KEY ACHIEVEMENTS

Project Milestones & Achievements

✓ Phase 1

Requirements analysis and architecture planning

✓ Phase 2

Database and backend foundation (13 models, 9 services)

✓ Phase 3

Core UI development (16 pages, responsive design)

✓ Phase 4

Business logic (request approval, borrow/return)

✓ Phase 5

QR unlock and hardware abstraction (GPIO support)

✓ Phase 6

Deployment preparation (scripts, systemd, kiosk setup)

✓ Phase 7

Final review and UAT (all tests passed)

✓ Total

7 phases, 156 files, 15 use cases, 100% complete

✓ All Phases Complete • Ready for Demo

TECHNOLOGY STACK SUMMARY

Technology Stack & Architecture

Backend

ASP.NET Core 8.0

Frontend

Razor Pages with Bootstrap CSS

Database

SQLite with Entity Framework Core

Authentication

Cookie-based with BCrypt hashing

Hardware

Raspberry Pi GPIO abstraction layer

Deployment

systemd service on Raspberry Pi OS 64-bit

UI Framework

Bootstrap 5 for responsive design

Validation

HTML5 client-side + server-side validation

Modern • Secure • Scalable Stack

By Ali
Fullstack
Integrator

KNOWN LIMITATIONS

System Limitations & Constraints

Timezone

Uses UTC (configurable per deployment)

Single Location

One physical location per instance

Network Scope

Local LAN/WiFi only (not internet-facing)

Concurrency

Designed for small teams (< 100 concurrent users)

QR Distribution

Manual distribution (no email/SMS)

Authentication

Cookie-based only (no OAuth/LDAP)

Backup

Manual SQLite backup required

Database

SQLite suitable for small deployments (< 10,000 items)

⚠ Documented • ⚠ Manageable • ⚠ Scalable

FUTURE ENHANCEMENTS

Recommended Future Improvements

Email Notifications

For request approval/rejection

SMS Notifications

For overdue items

Barcode Scanning

As alternative to QR codes

Multi-Language Support

i18n internationalization

Analytics Dashboard

For usage statistics

Backup & Restore

Automated backup functionality

Multi-Location Support

For multiple sites/branches

OAuth/LDAP Integration

Enterprise authentication

Mobile App

For staff on-the-go

 Roadmap •  Scalability •  Enhancement

DEMO FLOW

Live Demo Walkthrough

Step 1

Admin login and create user

Step 2

Admin create item and assign locker

Step 3

Staff login and search items

Step 4

Staff request item

Step 5

Admin approve request and generate QR

Step 6

Locker QR unlock (mock hardware)

Step 7

Try same QR again (single-use enforcement)

Step 8

Staff return item

Step 9

Admin view overdue items

Step 10

Admin view audit logs

▶ 10 Steps • 15 Minutes • Full Walkthrough

By Maya
UAT + Frontend
Bug Hunter

HOW TO RUN LOCALLY

Local Development Setup

Command

cd SmartLocker/src/SmartLocker.Web && dotnet run

HTTP Access

http://localhost:5000

HTTPS Access

https://localhost:5001

Admin Credentials

Username: admin | Password: admin123

Staff Credentials

Username: staff | Password: staff123

Database

SQLite smartlocker.db (auto-created)

Pre-populated

with test data (users, items, lockers)

✓ Ready to Run • ✓ No Configuration • ✓ Test Data Included

By Rafi
Raspberry Pi/
Linux Engineer

RASPBERRY PI DEPLOYMENT

Production Deployment on Raspberry Pi

Publish Command

```
./scripts/publish-linux-arm64.sh
```

Install Command

```
sudo ./scripts/install-raspberrypi.sh
```

Start Service

```
sudo systemctl start smartlocker
```

HTTP Access

```
http://localhost:5000
```

mDNS Access

```
http://smartlocker.local (if mDNS configured)
```

Kiosk Setup

```
./scripts/chromium-kiosk-autostart.sh
```

✓ Automated • ✓ Secure • ✓ Production-Ready

By Rafi
Raspberry Pi/
Linux Engineer

GITHUB REPOSITORY

Public Repository & Access

Repository URL

`https://github.com/MustaqiimR/smartlocker-dotnet-raspberrypi`

Commit Hash

`1122a289f2e69e0808865845c62e168dc5736425`

Branch

`master`

Files

156 files committed

Clone Command

`git clone https://github.com/MustaqiimR/smartlocker-dotnet-raspberrypi.git`

Description

Research project joint venture between ISense ASV Sdn Bhd and UKM FTSM

Documentation

Complete with deployment guides and UAT checklists

✓ Public • ✓ Complete • ✓ Ready to Clone

By Ali
Fullstack
Integrator

PROJECT STATUS

Final Project Status & Readiness

✓ Development

100% Complete

✓ Testing

All UAT Passed

✓ Security

All Controls Verified

✓ Documentation

Complete & Comprehensive

✓ Deployment

Scripts Ready

✓ GitHub

Public Repository

✓ Demo Ready

10-Step Walkthrough

✓ Production Ready

Ready for Deployment

✓ PROJECT COMPLETE • ✓ READY FOR DEMO • ✓ PRODUCTION READY

SUPPORT & RESOURCES

Documentation & Support Resources

README .md

Project overview and quick start guide

DEPLOYMENT_GUIDE .md

Step-by-step deployment instructions

SECURITY_REVIEW .md

Security controls and compliance

UAT_CHECKLIST .md

Complete testing checklist

Phase Reports

7 detailed phase completion reports

GitHub Repository

Public repository with all code

Nginx Config

Optional reverse proxy configuration

mDNS Setup

Hostname configuration guide

 Comprehensive •  Well-Documented •  Easy to Follow

By Manus
Project
Team

THANK YOU

SmartLocker System - Complete

The SmartLocker system is now complete, tested, documented, and ready for deployment.

- ✓ All 15 Use Cases Implemented
 - ✓ Production-Ready Code
- ✓ Comprehensive Documentation
 - ✓ Deployment Scripts Ready
 - ✓ GitHub Repository Public

This research project demonstrates a complete IoT-enabled locker management system built with modern .NET technologies, designed for deployment on Raspberry Pi with local network access.

Research Project: ISense ASV Sdn Bhd & UKM FTSM
Repository: <https://github.com/MustaqimR/smartlocker-dotnet-raspberrypi>